

Font-Ensembled Vision-Text Representations for Homoglyph Detection

Seton Liu¹ · Hailey Boriel¹ · Valerie Xin¹ · Rafael Palacios^{2,3} · Miguel Domingos de Santana Wanderley⁴ · Guilherme Rinaldo⁴ · Amar Gupta^{1,5}

Abstract Attackers may create fraudulent business names and domains by replacing characters with look-alikes from other scripts in what is called a homoglyph attack. These visually deceptive strings often bypass human visual inspection and pose serious threats to website spoofing. Advances in detection via string similarity metrics show that these methods enhance classification and are simple to implement. However, they have significant edge cases which motivate the use of more nuanced detection methods. Text and image encoders produce information-rich embeddings that transfer well, but are often slower. Vision-language models (VLMs) offer a middle-ground by producing multi-modal embeddings at a relatively low latency. This paper introduces a collection of contrastively tuned VLM text and image embeddings for homoglyph

detection, and examines the marginal benefit of introducing image models specifically tuned on different text fonts. Main contributions of this work include a contrastive Siamese training framework for VLM encoders, an ensemble architecture combining string metrics with multimodal embeddings, and a complementary font-subset selection strategy. Evaluated on a homoglyph benchmark, our ensembles, comprising contrastively tuned Sigmoid Loss for Language Image Pre-Training (SigLIP) text and image encoders combined with string similarity metrics, achieve competitive receiver operating characteristic area under the curve (ROC-AUC) scores ranging from 0.9898 to 0.9918, with measured latencies of up to 27.4 milliseconds per pair under controlled settings. This offers options depending on latency needs, from a standalone text encoder to a high-performing multi-modal ensemble for more flexible budgets.

Keywords Homoglyph Detection · Contrastive Learning · Representation Learning · Vision-Language Models · Perceptual Similarity

¹ Department of Electrical Engineering and Computer Science, Massachusetts Institute of Technology, 77 Massachusetts Ave., Cambridge, MA 02139, USA

² Cybersecurity at MIT Sloan (CAMS), Massachusetts Institute of Technology, 77 Massachusetts Ave., Cambridge, MA 02139, USA

³ Institute for Research in Technology, Universidad Pontificia Comillas, Alberto Aguilera 23, 28015 Madrid, Spain

⁴ Instituto de Ciência e Tecnologia Itaú - ICTi, Av. Engenheiro Armando de Arruda Pereira 774, Jabaquara - São Paulo, São Paulo 04308-000, Brazil

⁵ AI Institute for Community-Engaged Research (AI-ICER), The University of Texas at El Paso, 500 West University Avenue, El Paso, TX 79968, USA

Corresponding author: Amar Gupta, agupta@mit.edu

Additional e-mails: Seton Liu: setliu22@mit.edu; Hailey Boriel: hail01@mit.edu; Valerie Xin: valxin@mit.edu; Rafael Palacios: palacios@mit.edu; Miguel Domingos de Santana Wanderley: miguel.wanderley@itau-unibanco.com.br; Guilherme Rinaldo: guilherme.rinaldo@itau-unibanco.com.br
ORCID iDs: Seton Liu: 0000-0002-8452-1131; Hailey Boriel: 0000-0003-2014-002X; Rafael Palacios: 0000-0002-8963-5074; Miguel Domingos de Santana Wanderley: 0009-0003-0039-9875; Guilherme Rinaldo: 0000-0002-0758-9802; Amar Gupta: 0000-0001-9306-1256

1 Introduction

Homoglyphs are characters from different writing systems that appear visually similar but correspond to distinct Unicode code points [1]. For example, the Latin character “a” and the Cyrillic letter resembling “a” are visually indistinguishable in many fonts despite representing different symbols. Such visual ambiguity enables *homoglyph attacks*, in which malicious actors impersonate trusted domain names by replacing characters with visually similar alternatives. Visual-deception attacks are used in phishing [2]; typosquatting affects domain names [3]; and impersonation attacks also occur on social networks [4]. Phishing volumes continue

to grow; in Q1 2025, the Anti-Phishing Working Group (APWG) observed 1,003,924 phishing attacks reported by its member companies, Global Research Partners, and public submissions to its reporting channels [5], the highest quarterly total since late 2023. Because the resulting strings appear nearly identical to legitimate identifiers, they can be difficult to distinguish visually and with simple string comparisons [1, 6].

More recent approaches explore learned representations for modeling similarity between strings. Neural embedding models map inputs to continuous vector spaces in which distances reflect semantic or perceptual relationships [7, 8]. Text encoders capture contextual and linguistic structure, while vision encoders extract visual features from images [9]. These modalities provide complementary information that can be exploited to detect visually deceptive identifiers. Contrastive learning has emerged as an effective strategy for adapting such embeddings, encouraging representations of visually similar strings to lie closer together while separating unrelated pairs [10].

Vision encoders are particularly appealing for homoglyph detection because the attacks themselves exploit visual similarity. By rendering strings as images, models can learn typographic cues that distinguish visually deceptive identifiers from unrelated text [1]. Prior work has also quantified the visual similarity of domain names for combating typosquatting abuse [11]. Rendering strings across fonts can change the visible appearance of characters, motivating the font-specific image models evaluated in this work. However, image-based approaches typically incur higher computational costs due to the rendering process, creating a trade-off between detection accuracy and latency.

In this work, we investigate contrastively trained Siamese encoders derived from pretrained vision-language models (VLMs) for homoglyph detection. Our approach utilizes both text and image embeddings to leverage complementary semantic and perceptual signals. We further construct ensemble models that combine similarity scores from string metrics and multiple font-specific image encoders. To efficiently select informative font subsets, we use complementary font-subset selection based on validation-set performance. Experiments show that combining contrastively tuned text and image encoders significantly improves detection performance over traditional string similarity metrics and pretrained embedding baselines, achieving a ROC-AUC of 0.9918 while maintaining practical inference latency. The resulting system supports multiple deployment configurations ranging from lightweight text-only models for high-throughput environments to higher-accuracy multi-modal ensembles.

The main contributions of this work are:

- A contrastive Siamese training framework that adapts pretrained VLM text and image encoders for homoglyph detection.
- An ensemble architecture combining symbolic string metrics with multimodal embeddings.
- A complementary font-subset selection strategy for image-based models.
- A flexible detection framework offering multiple accuracy and latency trade-offs for practical evaluation scenarios.

All experiments were conducted exclusively on public or synthetic datasets. No proprietary data, internal systems, or operational environments of any sponsoring organization were used. No real customer information was involved.

2 Related Work in Spoofed-Text Detection

2.1 String Similarity Metrics

String similarity metrics provide simple baselines for comparing identifiers. Levenshtein distance [12] measures the minimum number of insertions, deletions, or substitutions needed to transform one string into another; extensions such as Damerau-Levenshtein [13] additionally count transpositions. Approximate string-matching methods provide alternatives for comparing lexically similar strings [14]. Because many homoglyph attacks involve only a small number of substitutions, spoofed identifiers often retain small edit distances from their legitimate counterparts, making such methods a natural first line of defense. Rule-based systems further incorporate manually curated lists of visually confusable characters [15], and fuzzy matching libraries provide additional approximate matching scores for string comparison [16]. Liu *et al.* [11] proposed position-weighted, character-aware distance metrics that substantially improve typosquatting detection over standard Levenshtein alone.

Despite their efficiency, all of these methods capture only symbolic differences and therefore fail to model the perceptual similarity exploited in homoglyph attacks. Characters from different scripts may be visually identical yet correspond to distinct Unicode code points, causing spoofed strings to evade comparison [6]. Conversely, unrelated identifiers may share small edit distances, producing false positives. Simple thresholding thus yields poor precision and recall trade-offs. Rule-based systems remain limited by the Unicode substitutions explicitly covered by their rules [15, 17].

Despite their limitations, string similarity metrics remain valuable due to their low computational cost. Ensemble approaches have also been explored for homoglyph recognition [18]. Our approach combines traditional string similarity metrics with contrastively trained VLM encoders and uses a non-linear classifier to model their interactions.

2.2 Optical Character Recognition

Optical Character Recognition (OCR) converts rendered text images into symbolic representations, and has been explored as a preprocessing step for homoglyph detection [19]. Ginsberg and Yu [20] noted that OCR is poorly suited for this task: it is designed to map unknown inputs to a predefined character set, whereas homoglyph detection requires identifying all visually similar characters across the full Unicode space. In practice, OCR-based pipelines reintroduce the same limitations as string similarity approaches, since reconstructed text still must be compared symbolically. OCR performance can vary across visually ambiguous strings, and Unicode substitutions remain a challenge for symbolic processing [19, 17].

2.3 Embeddings for Similarity Scoring

Embedding models map inputs to continuous vector spaces in which related inputs can be compared [21]. Cosine similarity can be used to compare embedding vectors [22], and such comparisons integrate naturally with contrastive learning frameworks that encourage related inputs to lie closer together while separating unrelated pairs [10].

2.3.1 Text embeddings

Pretrained language models such as BERT [23] and word embedding models [24] map token sequences to dense representations, capturing semantic and contextual relationships from large text corpora [21]. For spoof detection, semantic similarity alone is insufficient. However, multi-modal models such as SigLIP are trained via joint image-text objectives [8]. We therefore evaluate whether their text encoders provide useful signals for spoof detection. Text encoders also offer lower inference latency than image-based approaches, making them attractive for high-throughput environments.

2.3.2 Image embeddings

Vision encoders can represent visual properties of text images [25]. Woodbridge *et al.* [1] demonstrated that

rendering strings as images and training a Siamese CNN enables detection of homoglyph attacks with substantially higher ROC-AUC than string similarity baselines. Subsequent work explored attention-based CNNs for homoglyph-domain detection [26] and generative augmentation for identifying homoglyph attacks [27]. Rendering strings across different fonts can also accentuate or suppress visual ambiguity between characters, motivating multi-font ensemble strategies presented in this work.

2.3.3 Vision Language Models (VLMs)

VLMs learn joint representations of images and text by aligning both modalities within a shared embedding space [7, 8]. Models including CoCa [28], FLAVA [29], and SigLIP [8] have demonstrated that multimodal pretraining on large image-caption datasets produces encoders that capture both visual and semantic information. A comprehensive survey of VLMs for visual tasks is provided by Zhang *et al.* [30]. Since VLM encoders align visual and textual representations, we evaluate whether their embeddings can support homoglyph detection. Our approach incorporates font-diverse image encoders alongside a contrastively tuned text encoder and string metrics. Homoglyphs have also been shown to induce cultural biases in text-to-image synthesis, highlighting the broader security implications of Unicode visual similarity [31].

2.4 Siamese Models

Siamese neural networks learn similarity relationships between paired inputs via two parameter-sharing branches that independently encode each element of a pair; the resulting representations are then compared with a similarity or distance function [32]. This design ensures both inputs are projected into a common representation space, enabling consistent comparisons. Metric-learning architectures have been applied to signature verification [32], face recognition [33], and one-shot image classification [34], where learning to judge pairwise similarity rather than to assign fixed class labels allows generalization to unseen identities. Woodbridge *et al.* [1] extended this paradigm to homoglyph detection, training a small Siamese CNN on rendered text images with contrastive loss; Vinayakumar and Soman [6] further applied Siamese recurrent networks directly on raw strings. In this work, we adopt Siamese architectures for both text and image encoders derived from pretrained VLMs, training with contrastive objectives and integrating the resulting similarity scores within an ensemble classifier.

2.5 Contrastive Learning

Contrastive learning trains encoders by attracting representations of positive pairs and repelling negative pairs. Landmark methods include SimCLR [35], which showed that strong data augmentation and a nonlinear projection head substantially improve self-supervised visual representations, and Contrastive Predictive Coding [36] and Supervised Contrastive Learning [37], which extended the paradigm to sequential prediction and to supervised settings, respectively.

A margin-based contrastive objective is adopted in the spirit of Hadsell *et al.* [10], adapting the classical contrastive formulation to cosine similarity with separate positive and negative margins. Given a pair (z_1, z_2) with label $y \in \{0, 1\}$, the ℓ_2 -normalized embeddings are found, and cosine similarity $c = \langle \hat{z}_1, \hat{z}_2 \rangle \in [-1, 1]$ is computed. Label-dependent margin constraints are also enforced via squared hinge penalties, positive pairs ($y = 1$) are encouraged to satisfy $c \geq m_{\text{pos}}$, and negative pairs ($y = 0$) are encouraged to satisfy $c \leq m_{\text{neg}}$:

$$\mathcal{L}_{\text{pos}}(c) = \max(0, m_{\text{pos}} - c)^2, \quad (1)$$

$$\mathcal{L}_{\text{neg}}(c) = \max(0, c - m_{\text{neg}})^2. \quad (2)$$

A strict gap $m_{\text{pos}} = m_{\text{neg}} + \Delta$ with $\Delta > 0$ creates a clear band of separation. To account for asymmetric difficulty, positive and negative penalties are weighted by w_{pos} and w_{neg} respectively, giving the per-example loss

$$\mathcal{L}(z_1, z_2, y) = w_{\text{pos}} y \mathcal{L}_{\text{pos}}(c) + w_{\text{neg}} (1 - y) \mathcal{L}_{\text{neg}}(c), \quad (3)$$

with the final objective being the mean over the batch.

2.6 Complementary Font-Subset Selection

We select complementary font subsets from a pool of 11 candidate font styles. DejaVu Sans is used as a fixed baseline and included in every configuration. Because the candidate pool is small, we enumerate all subsets containing $k \in \{2, 4\}$ additional fonts beyond the baseline. Rather than ranking fonts by standalone accuracy, we use a coverage-style objective that rewards complementary performance on the validation set. Such objectives are closely related to submodular subset selection under cardinality constraints [38, 39].

2.6.1 Discrete Objective

The discrete objective measures whether a candidate font improves predictions relative to the baseline font.

For each example x , the contribution of font f is

$$g_f(x) = \begin{cases} +1 & \text{if } f \text{ correctly predicts } x \text{ and the base-} \\ & \text{line does not} \\ 0 & \text{if the font and the baseline have the,} \\ & \text{same correctness outcome} \\ -1 & \text{if } f \text{ misclassifies } x \text{ while the baseline} \\ & \text{is correct} \end{cases}$$

For a candidate subset S of additional fonts, we define

$$F(S) = \sum_{x \in \mathcal{D}_{\text{val}}} \max_{f \in S} g_f(x), \quad (4)$$

where $\mathcal{D}_{\text{val}}$ denotes the validation set. For each target cardinality k , we evaluate all candidate subsets and select

$$S^* = \arg \max_{S \subseteq V, |S|=k} F(S). \quad (5)$$

This objective favors subsets whose members correct complementary baseline errors while penalizing subsets for which every selected font worsens a baseline prediction.

3 Experimental Setup

3.1 Datasets

3.1.1 Overview

We use the synthetic paired-name dataset released by Woodbridge *et al.* [1], which is built from a large collection of web domains and augmented with homograph spoofs generated through thousands of ASCII and Unicode character-substitution rules. Each example consists of a **fraudulent_name**, a corresponding **real_name**, and a binary label **label** indicating whether the pair is a spoof (1) or not (0). Benign pairs are formed from two legitimate names, while spoof pairs are produced by applying character-level substitution rules. The training set contains 976,122 pairs (positive = 627,507). The full validation split contains 51,380 pairs, from which we randomly sampled 9,999 examples for our experiments (positive = 6,428). The test set contains 256,886 pairs (positive = 165,141). A representative positive example is (**okayplaiier**, **okayplayer**) with **label** = 1, while a representative negative example is (**genbook**, **getbot**) with **label** = 0.

3.1.2 Spoof Subtypes

We categorize each (real, fraudulent) pair into a set of spoofing subtypes that capture how the fraudulent string was constructed (Table 1). These subtypes provide an interpretable, bucketed view of common edit operations and visual confusability patterns. Because a single spoof may involve multiple manipulations, the subtype assignments are not mutually exclusive.

3.1.3 Text to Image Conversion

To generate image-based inputs, each string is rendered as a 224×224 image by drawing the Unicode-normalized text in a Unicode-supporting font on a black background. The rendering procedure selects the largest integer font size (`minimum size` = 1, `cap` = 1024) that fits entirely within the 4 pixel padding with no wrapping or clipping. When the optimal font size at native resolution is ≤ 10 , the text is rendered at up to $8\times$ resolution and downsampled to 224×224 to improve sharpness. All names are processed through the SigLIP vision model (`google/siglip-base-patch16-224`), and the resulting feature vectors are ℓ_2 -normalized.

3.1.4 External Test Data

As a source of real-world software-package typosquatting attacks, we use the curated Typosquatting Dataset from `ecosyste-ms` [40]. This dataset contains 143 confirmed malicious-package/target-package mappings collected from public security research and larger malicious-package databases. After removing entries that did not reflect true typosquatting behavior, we constructed negative examples by pairing unrelated legitimate packages.

For more large-scale evaluation, we also consider the process-name test dataset introduced by Woodbridge *et al.* which contains 307,521 examples (positive = 178,419) [1]. This dataset is constructed from legitimate operating-system process names drawn from NSRL `.exe` and `.dll` filenames, together with malicious look-alike variants generated through character-level spoofing transformations.

3.2 Model Architecture

Our detection framework consists of two primary stages: (1) contrastive training of Siamese encoders that learn representations of visually deceptive text, and (2) an ensemble inference pipeline that combines similarity signals from multiple modalities.

3.2.1 Siamese Encoder Training

As shown in Figure 1, each input string is encoded by a pretrained SigLIP encoder, then passed through a shared two-layer ReLU projection head. Cosine similarity between the projected embeddings is optimized using the margin-based contrastive objective, encouraging spoof pairs to be close and non-spoof pairs to be separated.

3.2.2 Finetuned Image and Text Encoders

Multiple image encoders are trained, one per font, by rendering each string as a 224×224 image and fine-tuning a SigLIP vision encoder via the Siamese contrastive framework. Font-specific models capture diverse typographic signals. A SigLIP text encoder is fine-tuned in the same way, operating directly on tokenized strings; its multimodal pretraining already encodes some visual information about text.

3.2.3 Ensemble Inference Pipeline

At inference (Figure 2), each pair is evaluated by font-specific image encoders, the fine-tuned text encoder, and traditional string similarity metrics. Their similarity scores are concatenated and passed to a gradient boosting classifier that learns a non-linear combination, capturing complementary information across perceptual, textual, and symbolic signals.

3.3 Hyperparameter Optimization

We performed hyperparameter optimization using Optuna with the Tree-Structured Parzen Estimator (TPE) sampler over 50 trials, each training for 5 epochs. We selected hyperparameters (Table 2) by maximizing validation ROC-AUC, since the loss weighting terms (e.g., w_{neg}) can increase the reported loss while still improving discriminative performance.

3.4 Final Training and Early Stopping

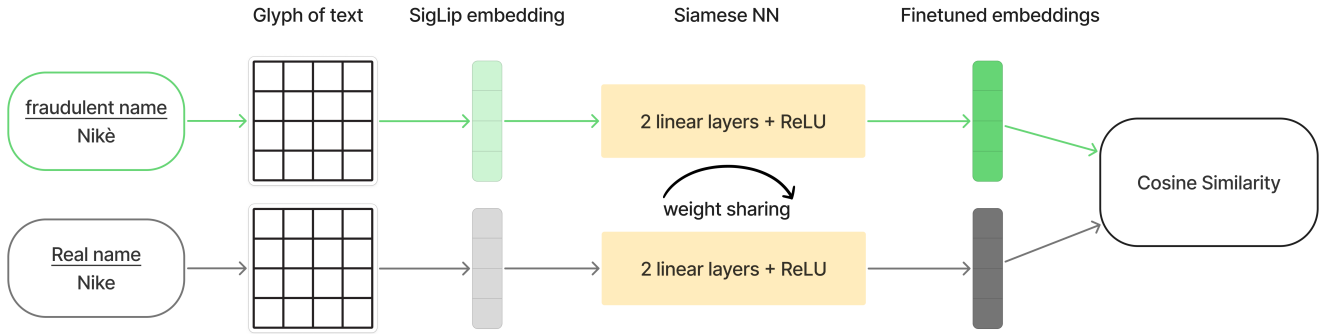
After hyperparameter selection, we fixed the chosen configuration and trained for up to 50 epochs with early stopping (patience = 5, min epochs = 25). Checkpoints were saved whenever the validation contrastive loss improved by at least 10^{-6} ; at the end of training, the best checkpoint was restored and evaluated on the test set once.

Table 1 Representative examples for each spoofing mechanism subtype (real $\rightarrow$ fraudulent)

Subtype	Example (real $\rightarrow$ fraud)	Notes
<i>Visual / Unicode confusability</i>		
Visual Confusable	mailce $\rightarrow$ mailce	Replace with any lookalike.
Non-ASCII	tcafek $\rightarrow$ tcāfēk	Replace with non-ASCII lookalike.
Mixed Script ¹	tophat $\rightarrow$ tophqt	Replace with different scripts.
Unicode	h4kurd $\rightarrow$ h4kȳrd	Replace with Unicode lookalike.
Multi-Char Confusable	mailce $\rightarrow$ rnailce	Replace with multiple lookalikes.
<i>Digit / symbol confusability</i>		
Digit Change	obaoba $\rightarrow$ o8aoba	Add digit(s).
Digit Affix	7learn $\rightarrow$ 7learn5	Add digit prefix/suffix.
Digit Substitution	10000w $\rightarrow$ 100p0w	Replace with any digit.
<i>Edit operations / formatting</i>		
Insertion	elgato $\rightarrow$ elgavto	Add character(s).
Affix	zazoom $\rightarrow$ zazoomd	Add prefix/suffix.
Repeated Character	jaleco $\rightarrow$ jaleeco	Add repeated character(s).
Deletion	viewdns $\rightarrow$ viewds	Delete character(s).
Separator Change	100njz $\rightarrow$ 100nj_z	Add/delete separator(s).
Hyphen Change	midomi $\rightarrow$ mido-i	Add/delete hyphen.
Substitution	hdfree $\rightarrow$ hefrees	Replace character(s).
Transposition	kinder $\rightarrow$ iknder	Swap adjacent characters.

¹ Due to rendering limitations, the Cyrillic small letter qa (U+051B) is replaced in this table with visually similar Latin *q*.

Training

**Fig. 1** Siamese training pipeline. Each input string is encoded by a pretrained SigLIP encoder, projected through a shared two-layer head, and trained with the margin contrastive objective

3.5 Performance Metrics

We report ROC-AUC, accuracy, precision, and recall. **ROC-AUC** summarizes threshold-independent discriminative performance and is insensitive to class imbalance [41]. **Accuracy** is $(TP + TN) / (TP + TN + FP + FN)$. **Precision** is $TP / (TP + FP)$. **Recall** is $TP / (TP + FN)$.

4 Experiments

4.1 Font Selection

Fonts were first screened based on their ability to reliably render Cyrillic and other Unicode characters appearing in the dataset. This filtering step produced 11 candidate fonts: Arimo, Charis SIL, Cousine, Doulos SIL, Exo 2, FreeSerif, Gentium Plus, Libre Franklin, Noto Sans, Unifont, and Vollkorn.

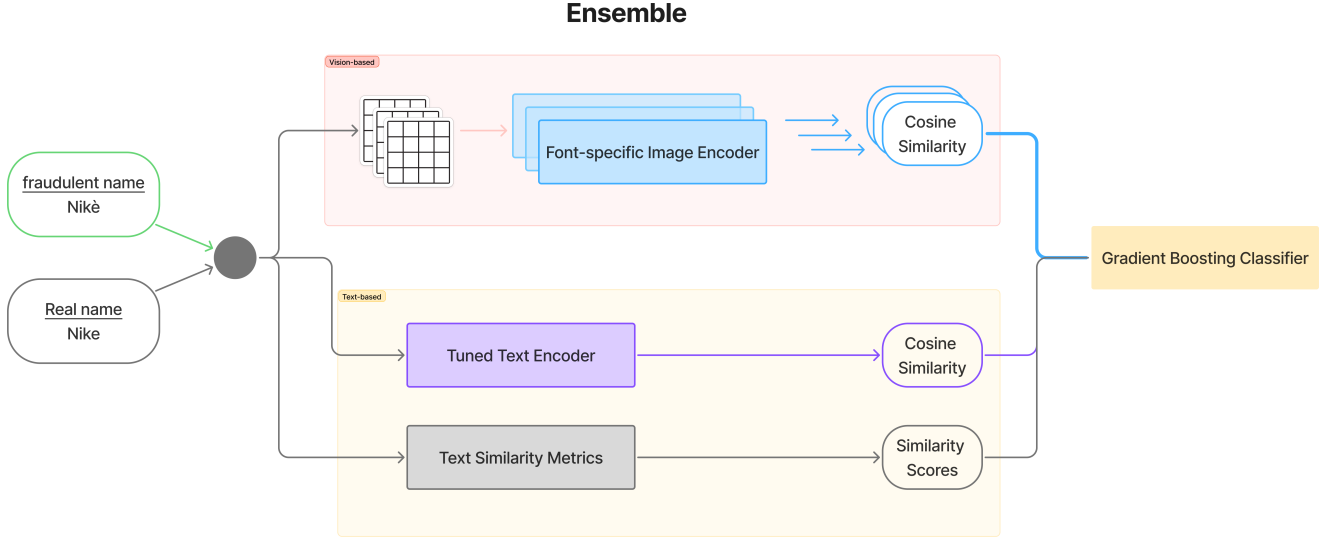


Fig. 2 Ensemble inference pipeline (Total 3F). String pairs are evaluated by font-specific image encoders, a text encoder, and string metrics; scores are combined via gradient boosting

Table 2 Optuna hyperparameter search space for the two-margin cosine contrastive objective.

Hyperparameter	Search range / choices
Learning rate (η)	$[10^{-5}, 10^{-3}]$ (log-uniform)
Batch size	{64, 128, 256, 512, 1024}
Internal layer size	{256, 512, 768, 1024}
Output dimension	{128, 256, 768}
Optimizer	{adam, adamw}
Weight decay	$[10^{-6}, 10^{-3}]$ (log-uniform)
m_{neg}	[0.75, 0.95] (uniform)
gap	[0.01, 0.2] (uniform)
m_{pos}	$\min(m_{\text{neg}} + \text{gap}, 0.99)$
w_{pos}	1.0 (fixed)
w_{neg}	[0.04, 25.0] (log-uniform)

We then applied the font-subset selection algorithm to font-specific predictions on the validation set to select representative font subsets. DejaVu Sans was used as the base font and included in every combination. The selected three-font subset was DejaVu Sans, Unifont, and Gentium Plus (Figure 3). The selected five-font subset was DejaVu Sans, Unifont, Cousine, Doulos SIL, and Libre Franklin.

4.2 Hyperparameter Optimization

Per-font optimal hyperparameters selected by Optuna are reported in Appendix A. w_{neg} varied substantially across fonts, suggesting that the difficulty of negative pairs depends on rendering characteristics.

5 Results and Discussion

Disclaimer: this research is purely academic and is based exclusively on public or synthetic data. It does not reflect any production systems, internal practices, or strategic initiatives of any sponsoring institution. All reported results were obtained under controlled experimental evaluation settings and are intended solely for research purposes.

5.1 Comparison to SOTA

We compared the proposed approach against string similarity metrics, text embedding models, image encoders, and OCR-based baselines under controlled experimental evaluation settings. The reported results characterize benchmark performance and should not be interpreted as measurements of deployed or production-system behavior. Performance and system metrics are summarized in Table 3. Intermediate variants isolate individual contributions: the “Text” model combines string metrics with the fine-tuned text encoder, “Image” combines string metrics with a single-font image encoder, and “Total (5F)” is the full ensemble.

Figure 4 visualizes the latency-accuracy trade-off: the ensemble introduces additional cost from multi-font rendering and inference while improving ROC-AUC.

Table 4 reports individual feature performance using the forest classifier. Font-based cosine similarity features score highest, with multiple fonts achieving ROC-AUC > 0.98 . Text-based cosine similarity is also strong, followed by string similarity metrics.

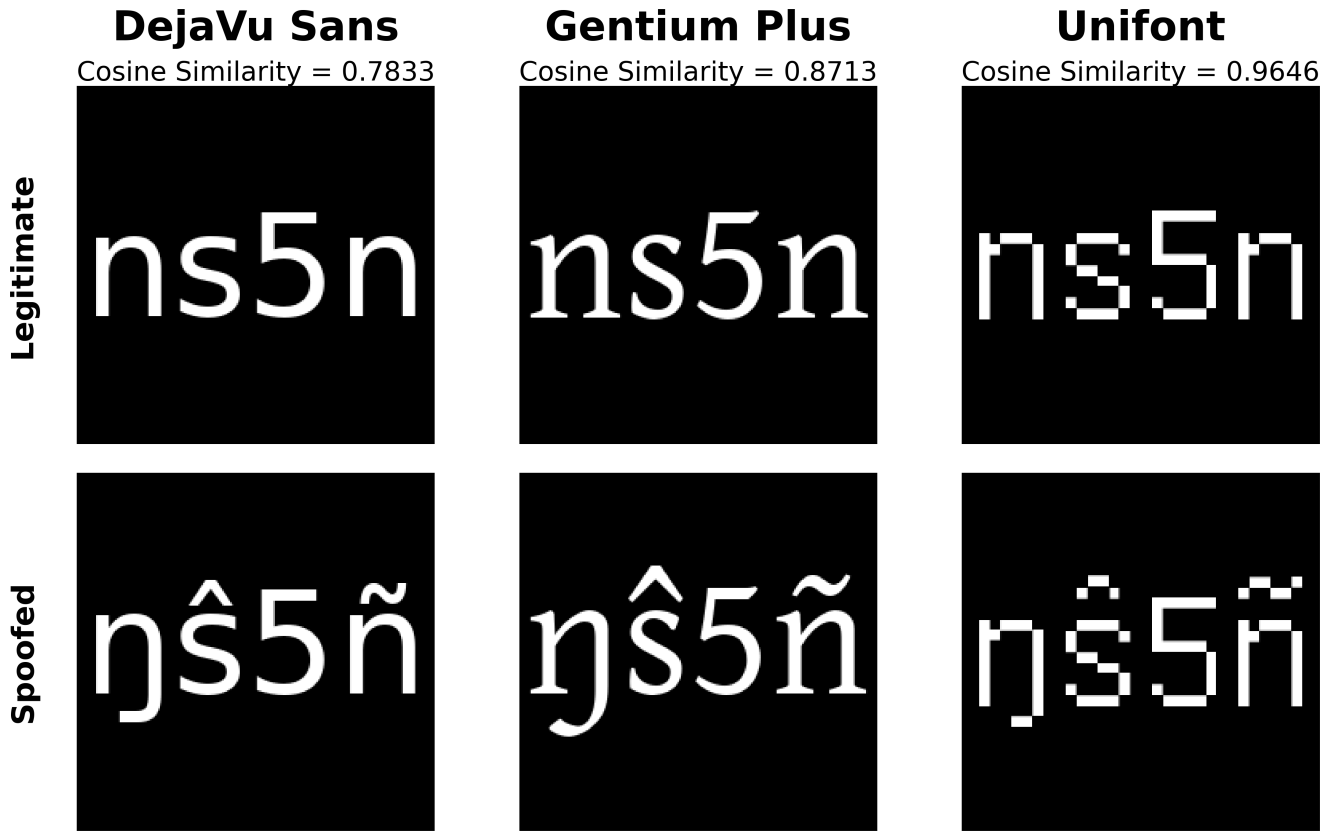


Fig. 3 The task of homoglyph detection is inherently visual and different fonts may accentuate distinct character-level similarities. Unifont renders the first U+014B glyph in a way that more closely resembles the “n” character it replaces (compared to DejaVu Sans or Gentium Plus), better reflecting the visual spoofing strategy

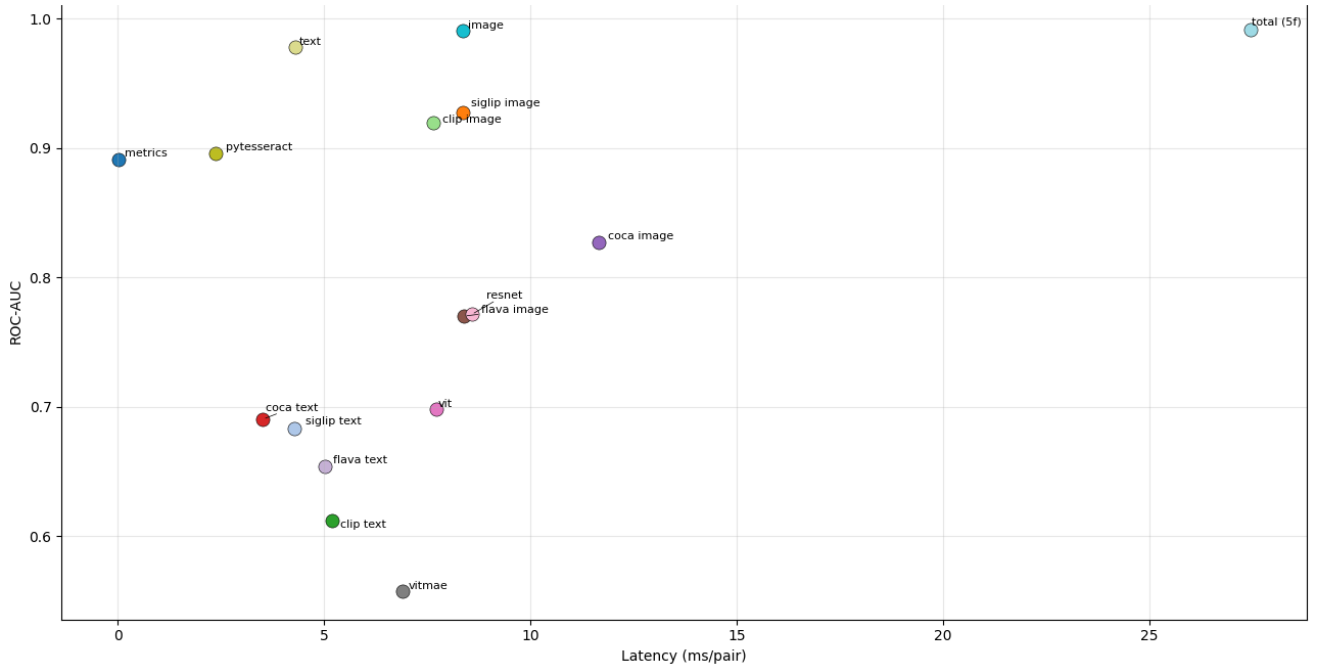


Fig. 4 Latency vs. ROC-AUC for all models. The proposed ensembles achieve the highest AUC, with increasing latency as additional font-specific image encoders are included

Table 3 Pairwise comparison of baseline methods and the proposed ensemble on the test set. Latency is the average inference time per pair. Throughput is reported in strings per second.

Model	Performance		System Metrics			
	AUROC	Acc.	Latency (ms/pair)	Throughput (strings/s)	CPU (%)	GPU (GB)
Baselines						
Metrics	0.8908	0.8209	0.0096	208773	0.18	0.000
SigLIP Text	0.6830	0.6600	4.2770	467.6	1.43	0.765
SigLIP Image	0.9275	0.8478	8.3547	239.4	6.59	0.765
CLIP Text	0.6118	0.5391	5.2023	384.4	1.61	0.572
CLIP Image	0.9197	0.8345	7.6484	261.5	6.57	0.574
CoCa Text	0.6903	0.6150	3.5164	568.8	1.26	0.579
CoCa Image	0.8272	0.7303	11.6420	171.8	6.10	0.579
FLAVA Text	0.6535	0.5473	5.0294	397.7	1.53	0.908
FLAVA Image	0.7700	0.6900	8.3928	238.3	6.51	0.908
ViT	0.6981	0.6061	7.7076	259.5	6.49	0.330
ResNet	0.7713	0.6787	8.5751	233.2	6.52	0.097
ViT-MAE	0.5572	0.5183	6.9070	289.6	6.18	0.329
PyTesseract	0.8955	0.8590	2.3786	840.8	0.88	0.008
Proposed Method						
Text	0.9782	0.9339	4.3010	465.0	2.76	0.765
Image	0.9903	0.9508	8.3608	239.2	9.22	0.354
Total (5F)	0.9918	0.9589	27.4397	72.9	4.28	1.522

Table 4 Performance of individual features on the test set using the forest classifier, ordered by ROC-AUC from highest to lowest. Cosine embedding features are reported after contrastive training; string similarity metrics are untrained input features.

Feature	ROC-AUC	Accuracy	Precision	Recall
Cosine (Gentium Plus)	0.9882	0.9482	0.9664	0.9526
Cosine (DejaVu Sans)	0.9880	0.9482	0.9677	0.9511
Cosine (Doulos SIL)	0.9879	0.9484	0.9676	0.9517
Cosine (Libre Franklin)	0.9862	0.9447	0.9617	0.9519
Cosine (Cousine)	0.9851	0.9418	0.9567	0.9525
Cosine (Unifont)	0.9832	0.9378	0.9556	0.9473
Cosine (Text)	0.9732	0.9272	0.9376	0.9500
Token Set Ratio	0.8372	0.7674	0.8187	0.8197
Levenshtein Distance Score	0.8136	0.8170	0.7930	0.9681
Partial Ratio	0.7638	0.7226	0.7375	0.8826

Table 5 shows that the full five-font ensemble attains a ROC-AUC of 0.9918 and an accuracy of 0.9589, exceeding all baselines. This result reflects the benefit of incorporating font-diverse image encoders alongside the text encoder and string metrics.

Ensemble ablations are summarized in Table 5. The modified five-font variant, which replaces Unifont with Gentium Plus, achieves a lower ROC-AUC than the standard five-font model despite Gentium Plus having a higher standalone ROC-AUC than Unifont. This confirms that the font selection procedure captures complementary information rather than simply ranking fonts by standalone performance. The three-font total model achieves ROC-AUC and accuracy only marginally below the five-font model, suggesting that a small set of complementary fonts captures most of the benefit at lower computational cost.

Figure 5 shows the confusion matrix for the full 5F ensemble.

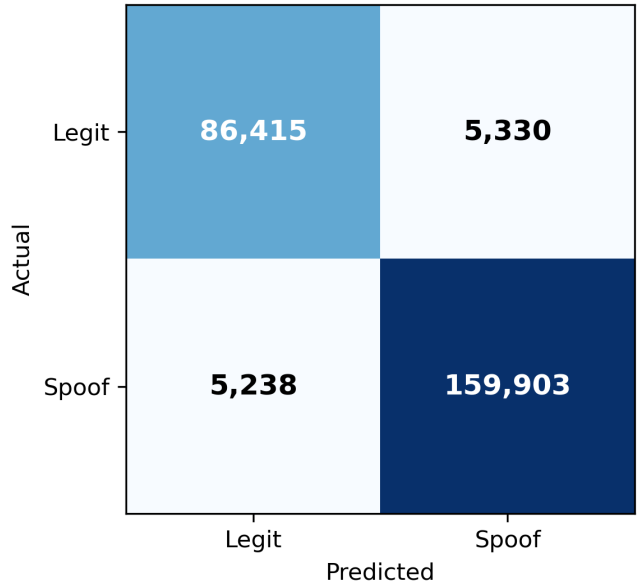
**Fig. 5** Confusion matrix for the 5F ensemble model

Table 5 Performance and composition of the ensemble models and ablation variants on the test set. A check mark indicates that the corresponding input type is included in the model, and blank cells indicate that it is not included. The columns “1F,” “3F,” “5F,” and “5F Mod.” denote the single-font, three-font, five-font, and modified five-font configurations, respectively. 5F Img. denotes the 5F configuration without text embeddings (only image embeddings and similarity metrics).

Ensemble	Performance				Inputs				
	ROC-AUC	Accuracy	Precision	Recall	Metrics	Text	1F	3F	5F 5F Mod.
Metrics	0.8908	0.8209	0.7924	0.9774	✓				
Text	0.9782	0.9339	0.9440	0.9537	✓	✓			
Image	0.9903	0.9508	0.9691	0.9539	✓		✓		
Small	0.9875	0.9465	0.9569	0.9601		✓	✓		
Total (1F)	0.9892	0.9491	0.9595	0.9614	✓		✓		
Medium	0.9898	0.9558	0.9614	0.9703		✓		✓	
Total (3F)	0.9912	0.9576	0.9626	0.9718	✓	✓		✓	
Large	0.9904	0.9562	0.9679	0.9638		✓			✓
Total (5F)	0.9918	0.9589	0.9677	0.9683	✓	✓		✓	
Total (5F Mod.)	0.9913	0.9571	0.9667	0.9666	✓	✓			✓
Total (5F Img.)	0.9900	0.9518	0.9608	0.9644	✓			✓	

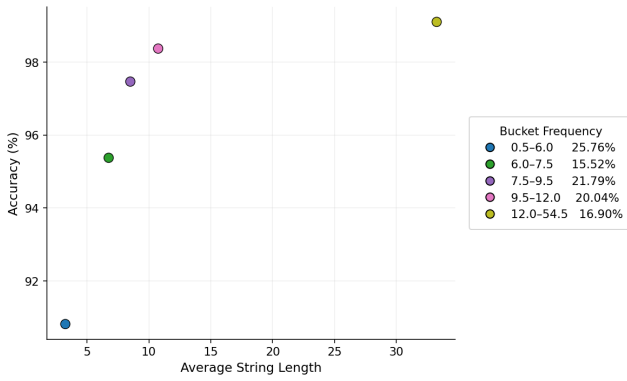


Fig. 6 Accuracy vs. average string length across all samples (five equal-size buckets; legend reports per-bucket prevalence)

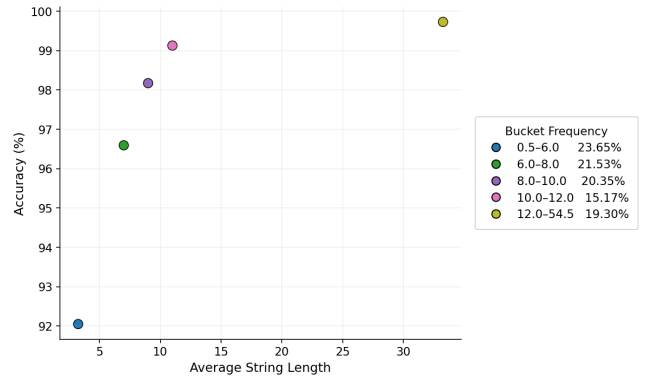


Fig. 7 Accuracy vs. average string length for spoofer samples only (`label=1`)

5.2 Error Analysis

We conducted an error analysis using the 5F ensemble. Figure 6 shows that performance declines for shorter strings: small edits produce high visual similarity in short names, making spoofer and non-spoofer pairs harder to distinguish. Dynamic font sizing helps on longer strings by preserving character proportions.

The same trend appears when considering spoofer samples only, as shown in Figure 7.

Figure 8 shows accuracy decreasing as the absolute length difference between real and fraudulent names grows. This is driven by the inherent decrease in cosine similarity for pairs with different lengths, amplified by dynamic font resizing. Critically, the dataset contains very few spoofer samples with absolute length differences greater than three, which is a significant gap given that real-world attacks may involve more substantial length changes.

Figure 9 plots subtype-specific miss rate against subtype frequency. Digit affix has the highest miss rate among the plotted subtypes; Table 6 shows that it

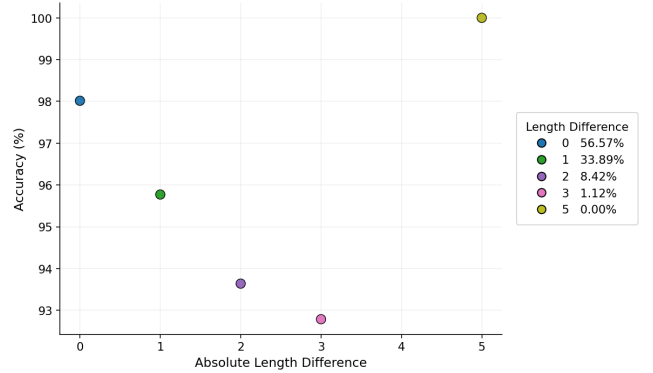


Fig. 8 Accuracy vs. absolute length difference between the fraudulent and real names for spoofer samples (`label=1`). The legend reports the prevalence of each bucket in the dataset. An outlier occurs at an absolute length difference of 5

also has the shortest average fraudulent and real string lengths among these subtypes.

The model performs well on non-ASCII spoofs, likely because multiple font renderings capture cross-script visual variation effectively. Most high-miss-rate subtypes involve string length changes, consistent with the length-difference analysis above.

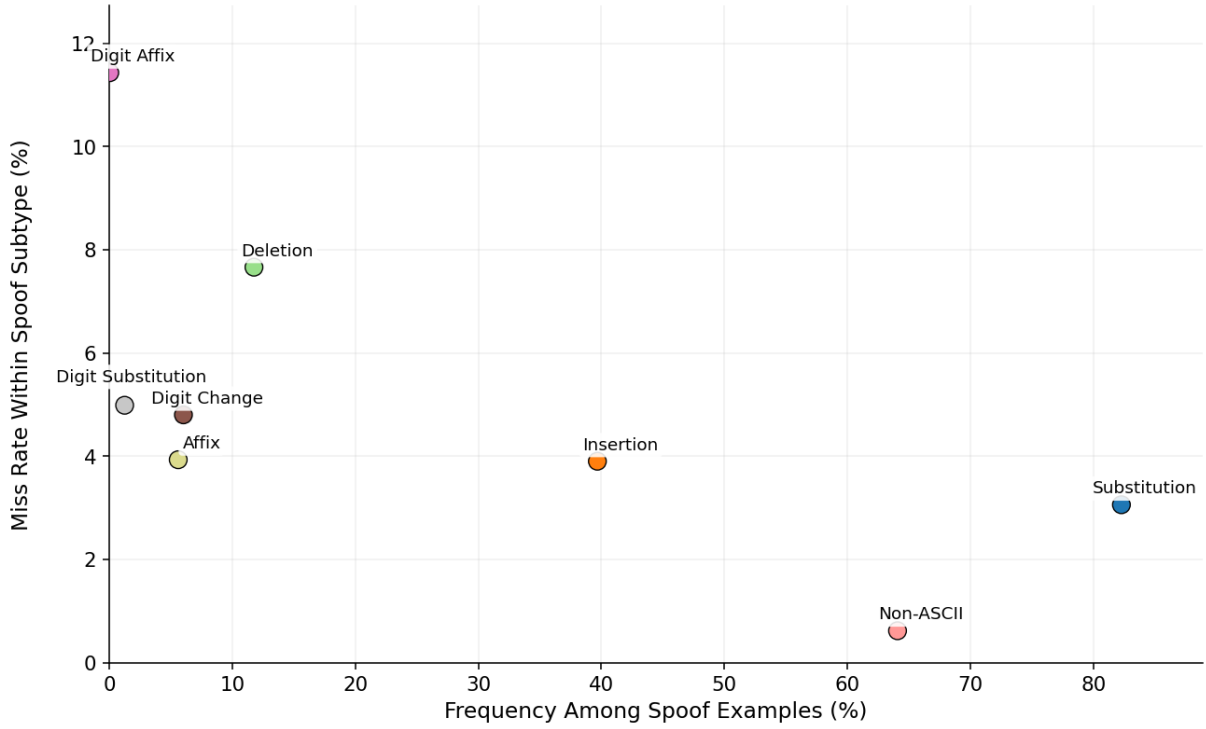


Fig. 9 Most impactful error types: representative spoof subtypes with the highest miss rates and/or highest frequencies among spoof samples (`label=1`)

Table 6 Summary statistics for the most impactful spoof subtypes. Figure 9 reports subtype-level miss rates and frequencies; this table reports error counts and length statistics for the same subtypes. Length statistics shown are for all spoofs of that subtype, not just misclassified samples. Examples of spoof subtypes are shown in Table 1.

Mechanism	Errors	Avg. Length (Fraud)	Avg. Length (Real)	Avg. Length Diff.
Digit Affix	8	5.97	5.81	1.04
Deletion	1488	9.55	9.60	0.55
Digit Substitution	103	8.19	8.19	0.00
Digit Change	478	9.88	9.44	0.53
Affix	364	8.42	7.22	1.30
Insertion	2556	10.01	8.97	1.07
Substitution	4155	9.55	9.21	0.42
Non-ASCII	653	9.39	9.11	0.34

5.3 Translational Performance

To evaluate generalization, we assess performance on the real-world ecosystems typosquatting dataset and the Woodbridge process-name dataset. On the ecosystems dataset, the full ensemble achieves the highest performance among the reported visual baselines and proposed models. On the Woodbridge process-name dataset, several visual baselines approach or fall below random performance, while the 5F ensemble maintains the highest ROC-AUC. ROC-AUC scores are summarized in Table 7. Per-font hyperparameters are reported in Appendix A (Table 8), and detailed precision, recall, and specificity under varying recall constraints are reported in Appendix B (Tables 9 and 10).

Table 7 ROC-AUC of baseline methods and the proposed ensemble on translational datasets.

Visual Model / Method	Ecosystems Dataset	Process-Name Dataset
CLIP	0.9521	0.2622
CoCa	0.8834	0.4617
FLAVA	0.9628	0.4141
ViT	0.8739	0.4602
ViT-MAE	0.6104	0.4874
ResNet	0.8831	0.4912
PyTesseract	0.5941	0.6057
AttentionCNN	0.5005	0.5293
Total (5F)	0.9766	0.7974

5.4 Limitations

The reported results are obtained under controlled experiments conducted exclusively on public or synthetic datasets. Thus, these results do not directly establish performance in deployed or production environments, where data distributions, infrastructure constraints, and adversarial behavior may differ.

Another central limitation arises from rendering strings onto a fixed-size canvas. Dynamic resizing avoids clipping for long strings but introduces scale differences when the real and spoofed names differ in length. Fixed-size and line-wrapped alternatives each carry their own trade-offs (Figure 10); we adopt dynamic resizing as the most broadly applicable strategy to mitigate the effect without compromising performance.

The three pairs in the ecosystems dataset that no font-ensemble model correctly classified at recall ≥ 0.975 , (**fairseq**, **fairsequence**), (**requests**, **requests-darwin-lite**), and (**triton**, **torchtriton**), all involve large length differences and exhibit low cosine similarity under every rendering strategy, suggesting that more robust handling of insertion/deletion-heavy spoofs requires approaches beyond the current framework.

6 Conclusions

We presented a font-ensembled vision-text detection framework for homoglyph attacks that combines contrastively fine-tuned SigLIP text and image encoders with traditional string similarity metrics. Our contrastive Siamese training framework adapts pretrained VLM encoders to the homoglyph detection task, and a complementary font-subset selection strategy identifies informative font combinations for multi-font image encoding. The resulting ensemble achieves a ROC-AUC of 0.9918 on the Woodbridge benchmark, with a measured latency of 27.4 ms/pair under controlled experimental evaluation settings. Evaluated on two external datasets, a real-world typosquatting dataset and the Woodbridge process-name dataset, the ensemble consistently outperforms all baselines, including standalone VLM text and image encoders, OCR-based approaches, and string similarity metrics.

Error analysis reveals that performance degrades for short strings and pairs with large absolute length differences, both of which reduce cosine similarity regardless of rendering strategy. These failure modes point to a need for complementary containment-based or length-aware signals. The framework naturally scales with the font pool and supports multiple deployment configurations, from a lightweight standalone text encoder for

high-throughput settings to the full five-font ensemble for maximal accuracy.

Future directions include improved rendering strategies for long strings, expanded multilingual training data, adaptive font selection at inference time, potential integration with broader detection pipelines in future research, and application to related identity spoofing tasks such as social media impersonation and software package typosquatting.

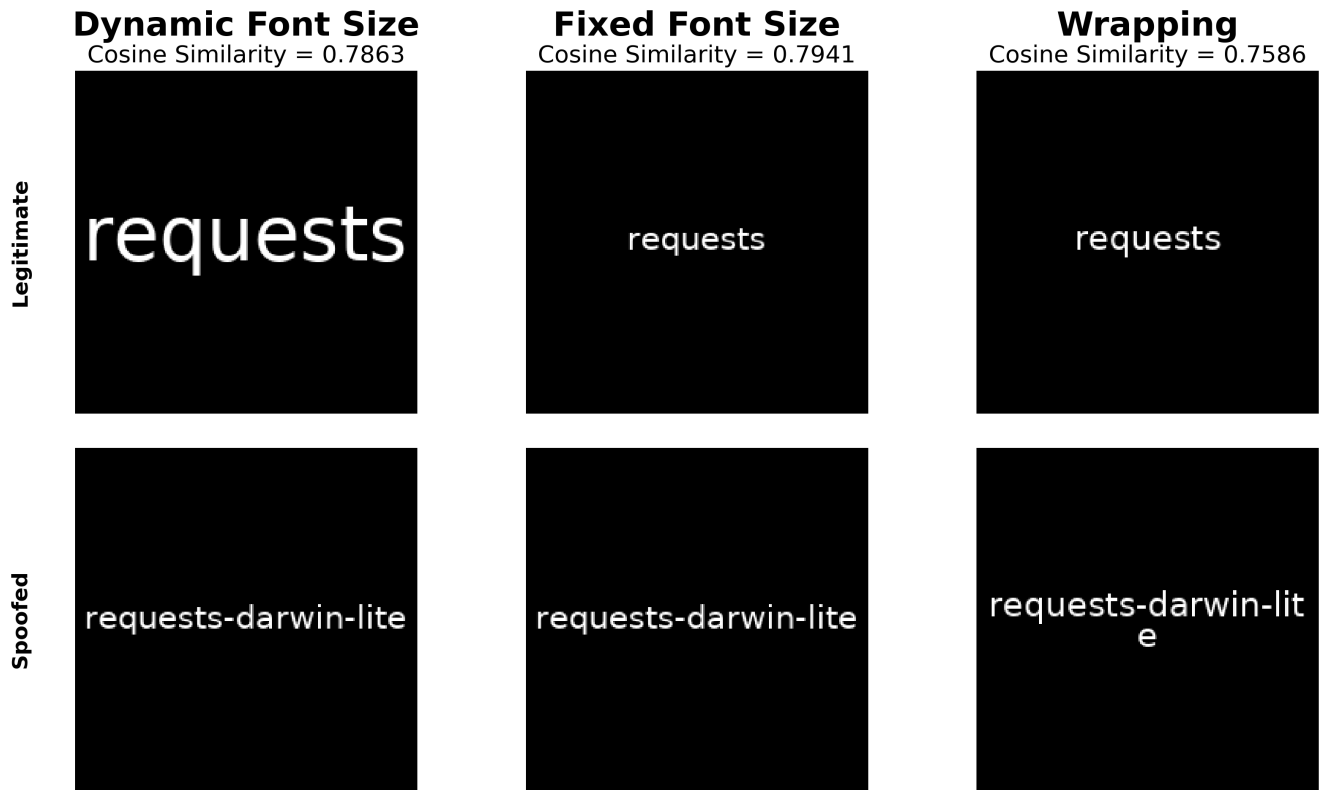


Fig. 10 Comparison of text-rendering strategies on a fixed canvas and the resulting cosine similarity between SigLIP embeddings

Statements and Declarations

Funding. Funding for this research was provided by Itaú Unibanco. The sponsor had no involvement in data collection, modeling decisions, or experimental evaluation.

Author contributions. Seton Liu and Hailey Boriel contributed equally to this work. All authors contributed to the study conception and design. Material preparation and analysis were performed by Seton Liu, Hailey Boriel, and Valerie Xin. Amar Gupta supervised the work.

Competing interests. The authors have no competing interests.

Data availability. The datasets used in this study were obtained from publicly available online sources cited in the manuscript.

A Hyperparameter Optimization Results

Table 8 Optimal hyperparameters determined by Optuna per font/encoder, selected by validation ROC-AUC.

Hyperparameter	DejaVu Sans	SigLIP Text	Unifont	Gentium Plus	Cousine	Doulos SIL	Libre Franklin
Learning rate (η)	4.6401×10^{-5}	1.6330×10^{-5}	5.1055×10^{-5}	5.3818×10^{-5}	2.9902×10^{-5}	3.1041×10^{-5}	2.9902×10^{-5}
Batch size	512	64	128	512	256	64	256
Internal layer size	768	1024	1024	768	768	512	768
Output dimension	768	768	256	768	768	256	768
Optimizer	adam	adam	adamw	adamw	adam	adamw	adam
Weight decay	2.0814×10^{-6}	2.2918×10^{-6}	1.4594×10^{-6}	3.8252×10^{-6}	3.3494×10^{-6}	2.9553×10^{-4}	3.3494×10^{-6}
m_{neg}	0.8700	0.7798	0.9081	0.9134	0.9226	0.9099	0.9226
gap	0.1031	0.1267	0.1013	0.0542	0.1613	0.1827	0.1613
m_{pos}	0.9732	0.9065	0.9900	0.9676	0.9900	0.9900	0.9900
w_{pos}	1.0	1.0	1.0	1.0	1.0	1.0	1.0
w_{neg}	1.7732	1.5085	6.9464	1.8854	2.5005	4.2812	2.5005

B Translational Performance Details

Table 9 Precision, recall, and specificity under varying recall constraints on the ecosystems typosquatting dataset. Thresholds maximize precision subject to the recall requirement.

Recall $\geq$	Metric	Text Metrics	1F	3F	5F
0.975	Accuracy	0.0623	0.9619	0.9605	0.9447
	Precision	0.0623	0.6238	0.6150	0.5304
	Recall	1.0000	0.9776	0.9776	0.9776
	Specificity	0.0000	0.9608	0.9593	0.9425
0.950	Accuracy	0.0623	0.9805	0.9912	0.9916
	Precision	0.0623	0.7805	0.9078	0.9143
	Recall	1.0000	0.9552	0.9552	0.9552
	Specificity	0.0000	0.9821	0.9936	0.9940
0.925	Accuracy	0.9888	0.9907	0.9930	0.9940
	Precision	0.8986	0.9254	0.9612	0.9618
	Recall	0.9254	0.9254	0.9254	0.9403
	Specificity	0.9931	0.9950	0.9975	0.9975
0.900	Accuracy	0.9921	0.9930	0.9930	0.9926
	Precision	0.9680	0.9685	0.9612	0.9758
	Recall	0.9030	0.9179	0.9254	0.9030
	Specificity	0.9980	0.9980	0.9975	0.9985

Table 10 Precision, recall, and specificity under varying recall constraints on the Woodbridge process-name dataset. Thresholds maximize precision subject to the recall requirement.

Recall $\geq$	Metric	Text Metrics	1F	3F	5F
0.975	Accuracy	0.5802	0.6609	0.6677	0.6791
	Precision	0.5802	0.6354	0.6403	0.6487
	Recall	1.0000	0.9750	0.9750	0.9750
	Specificity	0.0000	0.2268	0.2429	0.2702
0.950	Accuracy	0.5802	0.6842	0.6832	0.7015
	Precision	0.5802	0.6578	0.6570	0.6716
	Recall	1.0000	0.9500	0.9502	0.9500
	Specificity	0.0000	0.3169	0.3144	0.3581
0.925	Accuracy	0.5802	0.6952	0.6923	0.7103
	Precision	0.5802	0.6725	0.6701	0.6856
	Recall	1.0000	0.9251	0.9250	0.9250
	Specificity	0.0000	0.3775	0.3706	0.4137
0.900	Accuracy	0.5802	0.7015	0.6992	0.7147
	Precision	0.5802	0.6842	0.6824	0.6966
	Recall	1.0000	0.9014	0.9007	0.9002
	Specificity	0.0000	0.4251	0.4208	0.4583

References

1. J. Woodbridge, H.S. Anderson, A. Ahuja, D. Grant, in *2018 IEEE Security and Privacy Workshops (SPW)* (2018), pp. 22–28. <https://doi.org/10.1109/SPW.2018.00012>
2. R. Dhamija, J.D. Tygar, M. Hearst, in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Association for Computing Machinery, New York, NY, USA, 2006), CHI '06, p. 581–590. <https://doi.org/10.1145/1124772.1124861>
3. A. Linari, F. Mitchell, D. Duce, S. Morris, in *Proceedings of the WebSci'09: Society On-Line* (2009)
4. O. Goga, G. Venkatadri, K.P. Gummadi, in *Proceedings of the 2015 Internet Measurement Conference* (Association for Computing Machinery, New York, NY, USA, 2015), IMC '15, p. 141–153. <https://doi.org/10.1145/2815675.2815699>
5. Anti-Phishing Working Group, Phishing activity trends report: 1st quarter 2025. Tech. rep., Anti-Phishing Working Group (2025). URL https://docs.apwg.org/reports/apwg_trends_report_q1_2025.pdf
6. R. Vinayakumar, K.P. Soman, *ICT Express* **6**(1), 16 (2020). <https://doi.org/10.1016/j.icte.2019.05.002>
7. A. Radford, J.W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, I. Sutskever, in *Proceedings of the 38th International Conference on Machine Learning, PMLR 139* (PMLR, 2021), pp. 8748–8763. URL <https://proceedings.mlr.press/v139/radford21a.html>
8. X. Zhai, B. Mustafa, A. Kolesnikov, L. Beyer, in *2023 IEEE/CVF International Conference on Computer Vision (ICCV)* (2023), pp. 11,941–11,952. <https://doi.org/10.1109/ICCV51070.2023.01100>
9. F. Bordes, R.Y. Pang, A. Ajay, A.C. Li, A. Bardes, S. Petryk, O. Mañas, Z. Lin, A. Mahmoud, B. Jayaraman, M. Ibrahim, M. Hall, Y. Xiong, J. Lebensold, C. Ross, S. Jayakumar, C. Guo, D. Bouchacourt, H. Al-Tahan, K. Padthe, V. Sharma, H. Xu, X.E. Tan, M. Richards, S. Lavoie, P. Astolfi, R.A. Hemmat, J. Chen, K. Tirumala, R. Assouel, M. Moayeri, A. Talattof, K. Chaudhuri, Z. Liu, X. Chen, Q. Garrido, K. Ullrich, A. Agrawal, K. Saenko, A. Celikyilmaz, V. Chandra. An introduction to vision-language modeling. arXiv preprint arXiv:2405.17247 (2024). <https://doi.org/10.48550/arXiv.2405.17247>
10. R. Hadsell, S. Chopra, Y. LeCun, in *2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'06)*, vol. 2 (2006), pp. 1735–1742. <https://doi.org/10.1109/CVPR.2006.100>
11. T. Liu, Y. Zhang, J. Shi, Y. Jing, Q. Li, L. Guo, in *MILCOM 2016 - 2016 IEEE Military Communications Conference* (2016), pp. 770–775. <https://doi.org/10.1109/MILCOM.2016.7795422>
12. V.I. Levenshtein, *Soviet Physics Doklady* **10**(8), 707 (1966)
13. F.J. Damerau, *Communications of the ACM* **7**(3), 171 (1964). <https://doi.org/10.1145/363958.363994>
14. K.P. Kalyanathaya, D. Akila, G. Suseendren, *Journal of Computational Information Systems* **15**(3), 26 (2019)
15. N. Roshanbin, J. Miller, in *Web Information System Engineering – WISE 2011, Lecture Notes in Computer Science 6997*, ed. by A. Bouguettaya, M. Hauswirth, L. Liu (Springer Berlin Heidelberg, Berlin, Heidelberg, 2011), pp. 1–14. https://doi.org/10.1007/978-3-642-24434-6_1
16. N. Elmobark, *International Journal of Environment, Engineering and Education* **7**(1), 48–60 (2025). <https://doi.org/10.55151/ijeedu.v7i1.188>
17. A.M. Almuhaideb, N. Aslam, A. Alabdullatif, S. Altamimi, S. Alothman, A. Alhussain, W. Aldosari, S.J. Alsunaidi, K.A. Alissa, *Journal of Sensor and Actuator Networks* **11**(3) (2022). <https://doi.org/10.3390/jsan11030054>
18. M.T.H. Majumder, M.M. Rahman, A. Iqbal, M.S. Rahman, *Mathematical and Computational Applications* **25**(4), 71 (2020). <https://doi.org/10.3390/mca25040071>
19. K. Shcherbyna, *International Journal of Computer Science and Mobile Computing* **13**(6), 22–26 (2024). <https://doi.org/10.47760/ijcsmc.2024.v13i06.004>
20. A. Ginsberg, C. Yu, in *2018 1st International Conference on Data Intelligence and Security (ICDIS)* (2018), pp. 17–23. <https://doi.org/10.1109/ICDIS.2018.00010>
21. S. Selva Birunda, R. Kanniga Devi, in *Innovative Data Communication Technologies and Application* (Springer Singapore, Singapore, 2021), pp. 267–281. https://doi.org/10.1007/978-981-15-9651-3_23
22. R.D. Jha, C. Zhang, V. Shmatikov, J.X. Morris, in *Advances in Neural Information Processing Systems*, vol. 38, ed. by D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, N. Chen (2025), vol. 38, pp. 45,963–45,987. URL <https://openreview.net/forum?id=jiCLUPq5xv>
23. J. Devlin, M.W. Chang, K. Lee, K. Toutanova, in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (Association for Computational Linguistics, Minneapolis, Minnesota, 2019), pp. 4171–4186. <https://doi.org/10.18653/v1/N19-1423>
24. T. Mikolov, I. Sutskever, K. Chen, G.S. Corrado, J. Dean, in *Advances in Neural Information Processing Systems 26 (NIPS 2013)* (2013), pp. 3111–3119. URL https://proceedings.neurips.cc/paper_files/paper/2013/file/9aa42b31882ec039965f3c4923ce901b-Paper.pdf
25. P. Krishnan, K. Dutta, C. Jawahar, in *2016 15th International Conference on Frontiers in Handwriting Recognition (ICFHR)* (2016), pp. 289–294. <https://doi.org/10.1109/ICFHR.2016.0062>
26. A. Gupta, L.S. Tomar, R. Garg. GlyphNet: Homoglyph domains dataset and detection using attention-based convolutional neural networks. arXiv preprint arXiv:2306.10392 (2023). <https://doi.org/10.48550/arXiv.2306.10392>
27. J.S. Lee, G.P.D. Yam, J.H. Chan, in *2020 International Conference on Communications, Computing, Cybersecurity, and Informatics (CCCI)* (2020), pp. 1–6. <https://doi.org/10.1109/CCCI49893.2020.9256804>
28. J. Yu, Z. Wang, V. Vasudevan, L. Yeung, M. Seyedhosseini, Y. Wu, *Transactions on Machine Learning Research* (2022). URL <https://openreview.net/forum?id=Ee277P3AYC>
29. A. Singh, R. Hu, V. Goswami, G. Couairon, W. Galuba, M. Rohrbach, D. Kiela, in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2022), pp. 15,617–15,629. <https://doi.org/10.1109/CVPR52688.2022.01519>
30. J. Zhang, J. Huang, S. Jin, S. Lu, *IEEE Transactions on Pattern Analysis and Machine Intelligence* **46**(8), 5625 (2024). <https://doi.org/10.1109/TPAMI.2024.3369699>

31. L. Struppek, D. Hintersdorf, F. Friedrich, M. Brack, P. Schramowski, K. Kersting, *Journal of Artificial Intelligence Research* **78**, 1017 (2023). <https://doi.org/10.1613/jair.1.15388>
32. J. Bromley, I. Guyon, Y. LeCun, E. Säckinger, R. Shah, in *Advances in Neural Information Processing Systems 6* (1993), pp. 737–744. URL <https://proceedings.neurips.cc/paper/1993/hash/288cc0ff022877bd3df94bc9360b9c5d-Abstract.html>
33. F. Schroff, D. Kalenichenko, J. Philbin, in *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2015), pp. 815–823. <https://doi.org/10.1109/CVPR.2015.7298682>
34. G. Koch, R. Zemel, R. Salakhutdinov, in *ICML Deep Learning Workshop, vol. 2* (Lille, France, 2015). URL <https://www.cs.cmu.edu/~rsalakhu/papers/oneshot1.pdf>
35. T. Chen, S. Kornblith, M. Norouzi, G. Hinton, in *Proceedings of the 37th International Conference on Machine Learning, PMLR 119* (PMLR, 2020), pp. 1597–1607. URL <https://proceedings.mlr.press/v119/chen20j.html>
36. A. van den Oord, Y. Li, O. Vinyals. Representation learning with contrastive predictive coding. arXiv preprint arXiv:1807.03748 (2018). <https://doi.org/10.48550/arXiv.1807.03748>
37. P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, D. Krishnan, in *Advances in Neural Information Processing Systems 33 (NIPS 2020)* (2020), pp. 18,661–18,673. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/d89a66c7c80a29b1bdbab0f2a1a94af8-Paper.pdf
38. A. Krause, D. Golovin, in *Tractability: Practical Approaches to Hard Problems*, ed. by L. Bordeaux, Y. Hamadi, P. Kohli (Cambridge University Press, 2014), pp. 71–104. <https://doi.org/10.1017/CB09781139177801.004>
39. G.L. Nemhauser, L.A. Wolsey, M.L. Fisher, *Mathematical Programming* **14**(1), 265 (1978). <https://doi.org/10.1007/BF01588971>
40. ecosyste-ms. Typosquatting dataset. GitHub repository. URL <https://github.com/ecosyste-ms/typosquatting-dataset>. Accessed 12 June 2026
41. T. Fawcett, *Pattern Recognition Letters* **27**(8), 861 (2006). <https://doi.org/10.1016/j.patrec.2005.10.010>